

UNIT -2

Functions, Classes and Objects:

Introduction of Classes, Class Definition, Defining a Members, Objects, Access Control, Class Scope, Scope Resolution Operator, Inline functions, Memory Allocation for Objects, Static Data Members, Static Member Functions, Arrays of Objects, Objects as Function Arguments, Friend Functions.

Introduction of Class:

An object oriented programming approach is a collection of objects and each object consists of corresponding data structures and procedures. The program is reusable and more maintainable. The important aspect in oop is a **class** which has similar syntax that of structure.

class: It is a collection of data and member functions that manipulate data. The data components of class are called data members and functions that manipulate the data are called member functions.

It can also called as blue print or prototype that defines the variables and functions common to all objects of certain kind. It is also known as user defined data type or ADT (abstract data type) A class is declared by the keyword **class**.

Syntax: -

```
class class_name
{
    Access specifier :
        Variable declarations;

    Access specifier :
        function declarations;

};
```

Access Control:

Access specifier or access modifiers are the labels that specify type of access given to members of a class. These are used for data hiding. These are also called as visibility modes. There are three types of access specifiers

1. private
2. public
3. protected

1. Private:

If the data members are declared as private access then they cannot be accessed from other functions outside the class. It can only be accessed by the functions declared within the class. It is declared by the key word „**private**“.

2. public:

If the data members are declared public access then they can be accessed from other functions outside the class. It is declared by the key word „**public**“.

3.protected: The access level of protected declaration lies between public and private. This access specifier is used at the time of inheritance Note:-
If no access specifier is specified then it is treated by default as private
The default access specifier of structure is public where as that of a class is “private”

Example:

```
class
student
{
private : int roll; char
        name[30];
public:
    void get_data()
    { cout<<"Enter roll number and name":
      cin>>roll>>name;
    }
    void put_data()
    {
        cout<<"Roll number:"<<roll<<endl;
        cout<<"Name      ":"<<name<<endl;
    }
};
```

Object:-Instance of a class is called object.

Syntax: class_name

object_name; Ex: student s;

Accessing members:-dot operator is used to access members of class

Object-name.function-name(actual arguments);

Ex:

```
s.get_data();
s.put_data();
```

Note:

- 1.If the access specifier is not specified in the class the default access specifier is private
- 2.All member functions are to be declared as public if not they are not accessible outside the class.

Object:

Instance of a class is called as object.

Syntax:

Class_name object name;

Example:

student s;

in the above example s is the object. It is a real time entity that can be used

Write a program to read data of a student

```
#include<iostream> using
```

```
namespace std; class
```

```
student
```

```
{ private:
```

```
    int roll;
```

```
    char name[20];
```

```

public:
    void getdata()

    {cout<<"Enter Roll number:";
      cin>>roll;
      cout<<"Enter Name:"; cin>>name;
    }
    void putdata()
    {cout<<"Roll                               no:"<<roll<<endl;
      cout<<"Name:"<<name<<endl;
    } }; int
    main() {
        student s;
        s.getdata();
        s.putdata(); return
        0;
    }

```

Scope Resolution operator:

Scope: -Visibility or availability of a variable in a program is called as scope. There are two types of scope. i)Local scope ii)Global scope

Local scope: visibility of a variable is local to the function in which it is declared.

Global scope: visibility of a variable to all functions of a program

Scope resolution operator in “**::**” .

This is used to access global variables if same variables are declared as local and global

```

#include<iostream.h>

int a=5;

void main()
{
    int a=1;

    cout<<"Local a"<<a<<endl;
    cout<<"Global a"<<::a<<endl;

}

```

Class Scope:

Scope resolution operator(::**)** is used to define a function outside a class.

```

#include <iostream>
using namespace std;
class sample
{ public:

    void output(); //function declaration
};

```

```
// function definition outside the class
void sample::output() {
    cout << "Function defined outside the class.\n";

};

int main() { sample
    obj;
obj.output();
    return 0;
}
```

Output of program:

Function defined outside the class.

Write a program to find area of rectangle

```
#include<iostream.h> class
rectangle
{ int
L,B;
public:
    void get_data();
    void area();
};

void rectangle::get_data()
{ cout<<"Enter Length of rectangle";
    cin>>L; cout<<"Enter breadth of
    rectangle"; cin>>B;
}
int rectangle::area()
{ return L*B;
} int
main()
{ rectangle
r;

    r.get_data();
    cout<<"Area of rectangle is"<<r.area();
return 0;
}
```

INLINE FUNCTIONS:

Definition:

An inline function is a function that is expanded in line when it is invoked. Inline expansion makes a program run faster because the overhead of a function call and return is eliminated. It is defined by using key word “**inline**”

Necessity of Inline Function:

- One of the objectives of using functions in a program is to save some memory space, which becomes appreciable when a function is likely to be called many times.

➤ Every time a function is called, it takes a lot of extra time in executing a series of instructions for tasks such as jumping to the function, saving registers, pushing arguments into the stack, and returning to the calling function.

➤ When a function is small, a substantial percentage of execution time may be spent in such overheads. One solution to this problem is to use macro definitions, known as macros. Preprocessor macros are popular in C. The major drawback with macros is that they are not really functions and therefore, the usual error checking does not occur during compilation.

➤ C++ has different solution to this problem. To eliminate the cost of calls to small functions, C++

proposes a new feature called inline function. **General**

Form:

```
inline function-header  
{ function body;  
}
```

Eg:

```
#include<iostream.h> inline  
float mul(float x, float y)  
{  
    return (x*y);  
}  
inline double div(double p, double q)  
{ return (p/q);  
}  
int  
main()  
{  
    float a=12.345; float  
    b=9.82;  
    cout<<mul(a,b);  
    cout<<div(a,b); return  
    0;  
}
```

Properties of inline function:

1. Inline function sends request but not a command to compiler
2. Compiler may serve or ignore the request
3. If function has too many lines of code or if it has complicated logic then it is executed as normal function

Situations where inline does not work:

- A function that is returning value, if it contains switch, loop or both then it is treated as normal function.
- If a function is not returning any value and it contains a return statement then it is treated as normal function

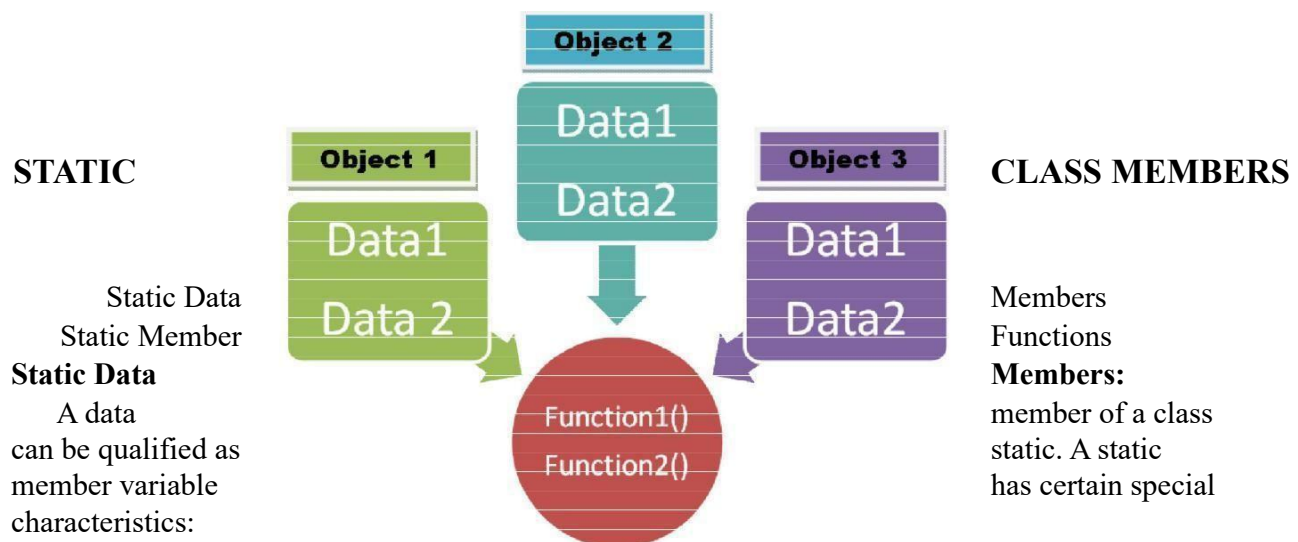
➤ If function contains static variables

then it is executed
as normal
function



If the inline function is declared as recursive function then it is executed as normal function.

Memory Allocation for Objects: Memory for objects is allocated when they are declared but not when class is defined. All objects in a given class uses same member functions. The member functions are created and placed in memory only once when they are defined in class definition



➤ It is initialized to zero when the first object of its class is created. No other initialization is permitted. Only one copy of that member is created for the entire class and is shared by all the objects of that class, no matter how

m

any objects are created.

➤ It is visible only within the class, but its lifetime is the entire program.

➤ Static data member is defined by keyword „static“ **Syntax:**

Data type class name::static_variable Name; Ex:

int item::count;

```
#include<iostream.h>
#include<conio.h> class
item
{ static int count; int
  number;
public:
  void getdata(int a)
  { number=a;
    count++;
  }
  void getcount()
  {
    cout<<"count is"<<count;
  } };
int item::count;//declearation
int main() { item a,b,c;
  a.getcount();
```

```

        b.getcount();
        c.getcount();
        a.getdata(100);
        b.getdata(200);
        c.getdata(300); cout<<"After
        reading data"; a.getcount();
        b.getcount();
        c.getcount();
return 0; } Output:
        count is 0
count is 0 count is
0 After reading data
count is 3 count is
3 count is 3

```

Static Member Functions

Like static member variable, we can also have static member functions. A member function that is declared static has the following properties:

A static function can have access to only other static members (functions or variables) declared in the same class.

A static member function is to be called using the class name (instead of its objects) as follows:

class-name :: function-name;

```

#include<iostream.h>
class test
{ int code; static int
    count;
public:
    void setcode()
    { code=++count;
    }
    void showcode()
    {
        cout<<"object number"<<code;
    }
    static void showcount()
    { cout<<"count"<<count;
    } }; int
test::count; int
main()
{ test t1,t2;
t1.setcode();
t2.setcode();
    test::showcount();
test t3; t3.setcode();
test::showcount();
t1.showcode();
t2.showcode();
t3.showcode(); return 0;
} Output: count 2 count
3 object number 1 object
number 2 object number
3

```

Arrays of Objects: Arrays of variables of type "class" is known as "Array of objects". An array of objects is stored inside the memory in the same way as in an ordinary array.

Syntax:

```
class class_name
{
    private:    data_type
members; public:
    data_type members;
    member functions;
};
```

Array of objects:

Class_name object_name[size];

Where size is the size of array Ex:

Myclass obj[10];

Write a program to initialize array of objects and print them

```
#include<iostream> using
```

```
namespace std; class
```

```
MyClass
```

```
{
```

```
int a;
```

```
public:
```

```
void set(int x)
```

```
{ a=x;
```

```
} int
```

```
get()
```

```
{ return a;
```

```
}
```

```
}; int
```

```
main()
```

```
{
```

```
MyClass obj[5];
```

```
for(int i=0;i<5;i++)
```

```
obj[i].set(i); for(int
```

```
i=0;i<5;i++)
```

```
cout<<"obj["<<i<<"].get():"<<obj[i].get()<<endl;
```

```
} Output:
```

```
obj[0].get():0
```

```
obj[1].get():1 obj[2].get():2
```

```
obj[3].get():3 obj[4].get():4
```

Objects as Function Arguments: Objects can be used as arguments to functions This can be done in three ways a. Pass-by-value or call by value

b. Pass-by-address or call by address

c. Pass-by-reference or call by reference

a.Pass-by-value – A copy of object (actual object) is sent to function and assigned to the object of called function (formal object). Both actual and formal copies of objects are stored at different memory locations. Hence, changes made in formal object are not reflected to actual object. write a program to swap values of two objects

write a program to swap values of two objects

```
#include<iostream.h>
using namespace std;
class sample2; class
sample1
{ int a;
public:
void
getdata
a(int
x);
    friend void display(sample1 x,sample2 y);
    friend void swap(sample1 x,sample2 y);
};
void sample1::getdata(int x)
{
    a=x;
}
class sample2
{ int b;
public:
    void getdata(int x);
    friend void display(sample1 x,sample2 y);

friend void swap(sample1 x,sample2 y);
};
void sample2::getdata(int x)
{ b=x;
}
void display(sample1 x,sample2 y)
{ cout<<"Data in object 1 is"<<endl;
    cout<<"a="<<x.a<<endl;
    cout<<"Data in object 2 is"<<endl;
    cout<<"b="<<y.b<<endl;
}
void swap(sample1 x,sample2 y)
{
int t; t=x.a;
    x.a=y.b;
    y.b=t; } int
main() { sample1 obj1;
sample2 obj2;
obj1.getdata(5);
obj2.getdata(15);
    cout<<"Before Swap of data between Two objects\n
"; display(obj1,obj2); swap(obj1,obj2);
    cout<<"after Swap of data between Two objects\n "; display(obj1,obj2);
```

```
}
```

Before Swap of data between Two objects

Data in object 1 is a=5 Data in object 2 is

b=15

after Swap of data between Two objects

Data in object 1 is a=5 Data in object 2

is b=15

b. Pass-by-address: Address of the object is sent as argument to function.

Here ampersand(&) is used as address operator and arrow (->) is used as de referencing operator.

If any change made to formal arguments then there is a change to actual arguments

write a program to swap values of two objects

```
#include<iostream.h>
```

```
using namespace std;
```

```
class sample2; class
```

```
sample1 { int a;
```

```
public:
```

```
    void getdata(int x); friend void
```

```
    display(sample1 x,sample2 y); friend void
```

```
    swap(sample1 *x,sample2 *y);
```

```
};
```

```
void sample1::getdata(int x)
```

```
{ a=x; } class
```

```
sample2 { int
```

```
b; public:
```

```
    void getdata(int x); friend void
```

```
    display(sample1 x,sample2 y);
```

```
    friend void swap(sample1 *x,sample2 *y);
```

```
};
```

```
void sample2::getdata(int x)
```

```
{ b=x; }
```

```
void display(sample1 x,sample2 y)
```

```
{ cout<<"Data in object 1 is"<<endl;
```

```
    cout<<"a="<<x.a<<endl;
```

```
    cout<<"Data in object 2 is"<<endl;
```

```
    cout<<"b="<<y.b<<endl;
```

```
}
```

```
void swap(sample1 *x,sample2 *y)
```

```
{
```

```
int t;
```

```
    t=x->a; x-
```

```
>a=y->b; y->b=t; } int
```

```
main() { sample1 obj1;
```

```
sample2 obj2;
```

```
obj1.getdata(5);
```

```
    obj2.getdata(15);
```

```
    cout<<"Before Swap of data between Two objects\n ";
```

```
    display(obj1,obj2); swap(&obj1,&obj2);
```

```
    cout<<"after Swap of data between Two objects\n "; display(obj1,obj2);
```

```
}
```

Before Swap of data between Two objects

Data in object 1 is a=5 Data in object 2
is b=15 after Swap of data between Two
objects
Data in object 1 is a=15
Data in object 2 is b=5

c.Pass -by-reference:A reference of object is sent as argument to function.

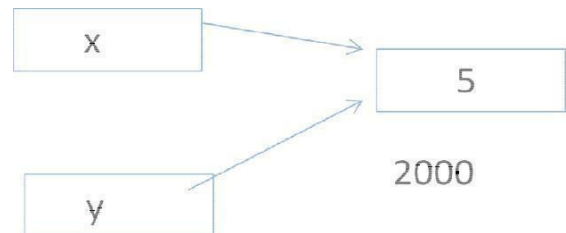
Reference to a variable provides alternate name for previously defined variable. If any change made to reference variable then there is a change to original variable.

A reference variable can be declared as follows

Datatype & reference variable =variable;

Ex:

```
int x=5;  
int &y=x;
```



Write a program to find sum of n natural numbers using reference variable

```
#include<iostream.h>  
using namespace std;  
int main() { int i=0;  
int &j=i; int s=0;  
int n; cout<<"Enter  
n:"; cin>>n;  
while(j<=n)  
{ s=s+i;  
i++;  
}  
cout<<"sum="<<s<<endl;  
}
```

Output: Enter
n:10
sum=55

write a program to swap values of two objects

```
#include<iostream.h>  
using namespace std;  
class sample2; class  
sample1  
{ int a;  
public:  
void getdata(int x);  
friend void display(sample1 x,sample2 y); friend  
void swap(sample1 &x,sample2 &y);  
};
```

```
void sample1::getdata(int x)
```

```

{ a=x;
}
class sample2
{ int b;
public:
    void getdata(int x);
    friend void display(sample1 x,sample2 y);
    friend void swap(sample1 &x,sample2 &y);
};
void sample2::getdata(int x)
{ b=x;
}
void display(sample1 x,sample2 y)
{ cout<<"Data in object 1 is"<<endl;
  cout<<"a="<<x.a<<endl;
  cout<<"Data in object 2 is"<<endl;
  cout<<"b="<<y.b<<endl;
}
void swap(sample1 &x,sample2 &y)
{
int t; t=x.a;
    x.a=y.b;
    y.b=t;
} int main()
{ sample1 obj1;
sample2 obj2;
obj1.getdata(5);
    obj2.getdata(15);
    cout<<"Before Swap of data between Two objects\n ";
    display(obj1,obj2); swap(obj1,obj2);
    cout<<"after Swap of data between Two objects\n ";
display(obj1,obj2); }

```

Output:

Before Swap of data between Two objects

Data in object 1 is a=5 Data

in object 2 is b=15

after Swap of data between Two objects Data

in object 1 is a=15

Data in object 2 is b=5

FRIEND FUNCTIONS: The private members cannot be accessed from outside the class. i.e.... a non member function cannot have an access to the private data of a class. In C++ a non member function can access private by making the function friendly to a class.

Definition:

A friend function is a function which is declared within a class and is defined outside the class. It does not require any scope resolution operator for defining . It can access private members of a class. It is declared by using keyword “**friend**” Ex:

class sample

```

{ int
x,y;
public:
    sample(int a,int b);
friend int sum(sample s);
};
sample::sample(int a,int b)
{
x=a;y=b; }
int sum(samples s)
{ int sum;
sum=s.x+s.y; return
0;
}
void main()
{
Sample obj(2,3); int
res=sum(obj);
cout<< "sum="<<res<<endl;
}

```

A friend function possesses certain special characteristics:

- It is not in the scope of the class to which it has been declared as friend.
- Since it is not in the scope of the class, it cannot be called using the object of that class. It can be invoked like a normal function without the help of any object.
- Unlike member functions, it cannot access the member names directly and has to use an object name and dot membership operator with each member name.
- It can be declared either in the public or private part of a class without affecting its meaning.
- Usually, it has the objects as arguments.

```

#include<iostream.h> class
sample
{ int a;
int b;
public:
    void setvalue()
    { a=25;
    b=40;
    }
    friend float mean(sample s);
};

```

```

float mean(sample s)
{ return float(s.a+s.b)/2.0;
} int
main()
{ sample
X;

```

```

        X.setvalue(); cout<<"Mean
        value="<<mean(X);
        return 0;
}

```

write a program to find max of two numbers using friend function for two different

```

classes #include<iostream>
using namespace std; class
sample2; class sample1
{ int x;
public:
    sample1(int a);
    friend void max(sample1 s1,sample2 s2)
};
sample1::sample1(int a)
{ x=a;
}
class sample2
{ int y;
public:
    sample2(int b);
    friend void max(sample1 s1,sample2 s2)
};
Sample2::sample2(int b)
{ y=b;
}
void max(sample1 s1,sample2 s2)
{
    If(s1.x>s2.y)
        cout<<"Data member in Object of class sample1 is larger "<<endl;
    else cout<<"Data member in Object of class sample2 is larger "<<endl;
}
void main()
{
    sample1 obj1(3);
    sample2 obj2(5);
    max(obj1,obj2); }

```

Write a program to add complex numbers using friend function

```

#include<iostream>
using namespace std;
class complex

{
float real,img; public:
    complex(); complex(float
    x,float y)
    friend complex add_complex(complex c1,complex c2);
};
complex::complex()

```

```

{ real=img=0;
}
complex::complex(float x,float y)
{ real=x;img=y;
}
complex add_complex(complex c1,complex c2)
{ complex
t;
    t.real=c1.real+c2.real;
    t.img=c1.img+c2.img;
    return t;
}
void complex::display ()
{ if(img<0)
    {img=-img; cout<<real<<"-i"<<img<<endl
    } else
    {
        cout<<real<<"i"<<img<<endl
    }
} int
main()
{
complex obj1(2,3); complex obj2(-4,-6);
complex obj3=add_complex(obj1,obj2);
    obj3.display();
    return 0;
}

```

Friend Class: A class can also be declared to be the friend of some other class. When we create a friend class then all the member functions of the friend class also become the friend of the other class. This requires the condition that the friend becoming class must be first declared or defined (forward declaration).

```

#include <iostream.h> class
sample_1
{ friend class sample_2;//declaring friend class int
a,b;
public:
    void getdata_1()
    { cout<<"Enter A & B values in class sample_1";
        cin>>a>>b;
    }

    void display_1()
    {
        cout<<"A="<<a<<endl; cout<<"B="<<b<<endl;
    }
}; class
sample_2
{ int c,d,sum;
    sample_1 obj1;
}

```

```

public: void
    getdata_2()
    { obj1.getdata_1();
      cout<<"Enter C & D values in class sample_2";
      cin>>c>>d;
    } void
    sum_2()
    { sum=obj1.a+obj1.b+c+d;
    }

    void display_2()
    { cout<<"A="<<obj1.a<<endl;
      cout<<"B="<<obj1.b<<endl;
      ; cout<<"C="<<c<<endl;
      cout<<"D="<<d<<endl;
      cout<<"SUM="<<sum<<endl;
    } }; int main()
{ sample_1 s1;
s1.getdata_1();
s1.display_1();
sample_2 s2; s2.getdata_2();
    s2.sum_2();
    s2.display_2();
}

```

Enter A & B values in class sample_1:1 2

A=1

B=2

Enter A & B values in class sample_1:1 2 3 4

Enter C & D values in class sample_2:A=1

B=2

C=3

D=4

SUM=10